SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence COURSE CODE: CSA1741

CO1-AT1-Analytical Problem Solving

Register Number	**192312174**
Name	**Pavithra S**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

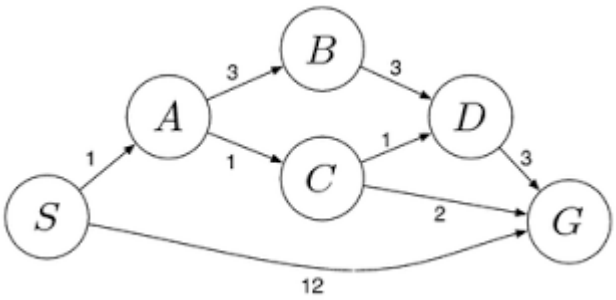
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I _____Pavithra S(192312174)_____ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Pavithra S

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1: Water Jug Problem

1. Problem Understanding

We have two empty jugs: one holds 4 gallons and the other holds 3 gallons. Neither jug has any markings. We must end with exactly 2 gallons of water in the 4-gallon jug.

Allowed actions

- Fill a jug completely from the pump.
- Empty a jug onto the ground.
- Pour water from one jug into the other, until the first is empty or the second is full.
- There is no other measuring tool.

Problem formulation

Component	Meaning for this problem
State	(x, y) where x = gallons in the 4-gallon jug (0 to 4) and y = gallons in the 3-gallon jug (0 to 3)
Initial state	(0, 0) – both jugs are empty
Goal state	(2, y) – the 4-gallon jug holds exactly 2 gallons (y can be anything)
Actions	Fill, empty and pour (6 rules, shown below)
Path cost	1 for each action, so a cheaper solution means fewer moves

2. Method Selection

Method chosen: Breadth-First Search (BFS). BFS is an uninformed search that checks all states 1 move away, then 2 moves away, and so on.

- Every move costs the same (1), so BFS is guaranteed to find the solution with the fewest moves.
- The problem is very small (at most 5 × 4 = 20 states), so BFS is fast and uses little memory.
- BFS remembers visited states, so the search never goes round in circles.

3. Solution Process

Production rules used by the program

No.	Condition	Action
1	x < 4	Fill the 4-gallon jug → (4, y)
2	y < 3	Fill the 3-gallon jug → (x, 3)
3	x > 0	Empty the 4-gallon jug → (0, y)
4	y > 0	Empty the 3-gallon jug → (x, 0)
5	x > 0 and y < 3	Pour 4-gallon into 3-gallon until 3-gallon is full or 4-gallon is empty
6	y > 0 and x < 4	Pour 3-gallon into 4-gallon until 4-gallon is full or 3-gallon is empty

Program in VS Code and its real output

```python
from collections import deque

CAP_X, CAP_Y = 4, 3      # 4-gallon jug (x) and 3-gallon jug (y)
GOAL = 2                 # we need exactly 2 gallons in the 4-gallon jug

def next_states(x, y):
    """Return every (rule, new_state) that can be applied to state (x, y)."""
    moves = []
    if x < CAP_X: moves.append(("Fill the 4-gallon jug from the pump", (CAP_X, y)))
    if y < CAP_Y: moves.append(("Fill the 3-gallon jug from the pump", (x, CAP_Y)))
    if x > 0:     moves.append(("Empty the 4-gallon jug on the ground", (0, y)))
    if y > 0:     moves.append(("Empty the 3-gallon jug on the ground", (x, 0)))
    t = min(x, CAP_Y - y)          # pour 4-gal -> 3-gal
    if t > 0:     moves.append(("Pour from 4-gallon jug into 3-gallon jug", (x - t, y + t)))
    t = min(y, CAP_X - x)          # pour 3-gal -> 4-gal
    if t > 0:     moves.append(("Pour from 3-gallon jug into 4-gallon jug", (x + t, y - t)))
    return moves

def solve():
    start = (0, 0)
    queue = deque([(start, [])])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
        if x == GOAL:
            return path
        for rule, state in next_states(x, y):
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [(rule, state)]))

print("WATER JUG PROBLEM (BFS)")
print("-" * 58)
print("Start state : (4-gal = 0, 3-gal = 0)")
```

```
WATER JUG PROBLEM (BFS)
----------------------------------------------------------
Start state : (4-gal = 0, 3-gal = 0)
Step 1: Fill the 4-gallon jug from the pump        -> (4, 0)
Step 2: Pour from 4-gallon jug into 3-gallon jug   -> (1, 3)
Step 3: Empty the 3-gallon jug on the ground       -> (1, 0)
Step 4: Pour from 4-gallon jug into 3-gallon jug   -> (0, 1)
Step 5: Fill the 4-gallon jug from the pump        -> (4, 1)
Step 6: Pour from 4-gallon jug into 3-gallon jug   -> (2, 3)
Goal reached: the 4-gallon jug now holds exactly 2 gallons.
```

4. Accuracy of Calculation

The step table below is copied from the real program output, so it is exact.

Step	Action	4-gal jug (x)	3-gal jug (y)
Start	Both jugs empty	0	0
1	Fill the 4-gallon jug from the pump	4	0
2	Pour from 4-gallon jug into 3-gallon jug	1	3
3	Empty the 3-gallon jug on the ground	1	0
4	Pour from 4-gallon jug into 3-gallon jug	0	1
5	Fill the 4-gallon jug from the pump	4	1

Step	Action	4-gal jug (x)	3-gal jug (y)
6	Pour from 4-gallon jug into 3-gallon jug (3-gallon takes only 2 more)	2	3

5. Interpretation of Results

- **Final answer:** the 4-gallon jug holds exactly 2 gallons after 6 moves.
- BFS checks the shortest paths first, so no solution with fewer than 6 moves exists.
- The trick is to use the 3-gallon jug as a measuring cup: 4 − 3 = 1 gallon, then 4 − (3 − 1) = 2 gallons.
- Because 3 and 4 share no common factor, any amount from 0 to 4 gallons can be measured with these jugs.

Question 2: Mars Rover – Intelligent Agent

1. Problem Understanding

A Mars Rover is a robot that travels on Mars, looks at rocks and soil, collects samples, analyses them and sends the results to Earth. We have to describe it as an intelligent agent. An agent is anything that senses its surroundings through sensors and acts on them through actuators.

We answer the five parts: (a) percepts, (b) environment, (c) actions, (d) performance measure and (e) best agent architecture.

2. Method Selection

Method chosen: the PEAS framework (Performance measure, Environment, Actuators, Sensors). It is the standard way to describe the task of an intelligent agent, and it covers all five sub-questions in an organised way.

PEAS item	Mars Rover
P – Performance	Useful samples analysed, area explored, safety, energy saved, data sent to Earth
E – Environment	Rocky, sandy and dusty surface of Mars with extreme weather
A – Actuators	Wheels, steering, robotic arm, drill, scoop, antenna, camera mast
S – Sensors	Cameras, spectrometer, temperature sensor, tilt sensor, wheel counters, radio receiver

3. Solution Process

(a) Percepts – what the rover senses

Percept	Sensor / source
Pictures of the ground, rocks and sky	Cameras (including stereo cameras)
Chemical and mineral content of soil and rock	Spectrometer and other science instruments
Obstacles and distance ahead	Obstacle / proximity sensors
Slope and tilt of the rover	Tilt sensor (gyroscope)

Percept	Sensor / source
Current position and distance travelled	Wheel counters and navigation system
Temperature, wind and dust	Weather sensors
Battery level and solar-panel power	Power sensors
Commands and updates from Earth	Radio receiver

(b) Operating environment

Property	Value	Reason
Observable	Partially	The rover sees only what its sensors can reach; hills and rocks hide things
Agents	Single agent	Only one rover works in the area (other rovers are far away)
Deterministic	Stochastic	Wheels may slip and dust storms are unpredictable
Episodic	Sequential	A wrong move now (for example, getting stuck) affects every later move
Static	Dynamic	Light, temperature and dust keep changing while the rover thinks
Discrete	Continuous	Position, speed and temperature take any value, not fixed steps
Known	Partly unknown	The terrain is new, so the rover must learn about it

A signal takes several minutes to reach Mars, so Earth cannot control the rover live. The rover must take most decisions by itself.

(c) Actions the rover can take

- Move forward or backward, turn left or right, and stop.
- Choose another path to avoid a rock or a steep slope.
- Take photographs of the surroundings.
- Use the robotic arm to touch, scoop or drill a sample.
- Put the sample inside the instruments and analyse it.
- Send the data to Earth through the antenna.
- Turn the solar panels towards the Sun, or go into sleep mode to save energy.

(d) How to evaluate the agent's performance

Performance measure	How it is measured
Science value	Number of useful, different samples collected and analysed

Performance measure	How it is measured
Exploration	Area covered or number of target places visited
Safety	Zero crashes, no tipping over, no hardware damage
Energy use	Work done per unit of battery used
Communication	Amount of correct data received on Earth
Lifetime	Number of days the rover keeps working

4. Accuracy of Calculation

This question needs no numbers, so accuracy here means that every PEAS part is complete and correct:

Part	Check
Percepts	8 percepts, each linked to a real sensor
Environment	7 standard properties, each with a reason
Actions	7 actions, all possible with the rover's actuators
Performance	6 measures covering science, safety, energy and lifetime

5. Interpretation of Results

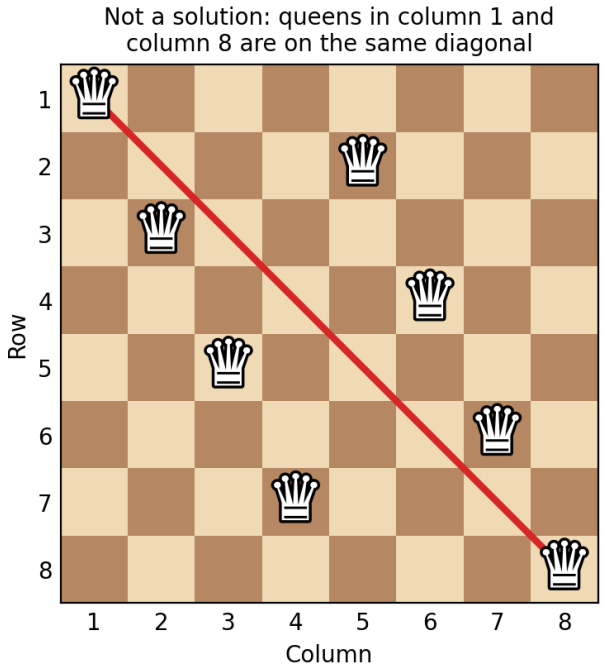
(e) Best agent architecture: goal-based + utility-based agent that can learn

Agent type	Is it suitable?
Simple reflex agent	No. It reacts only to the present percept. The rover is partially observable and needs memory.
Model-based reflex agent	Partly. It keeps a map and the rover's state, but it has no target to reach.
Goal-based agent	Yes. The rover has clear goals such as “reach that rock and analyse it”, and it plans a path.
Utility-based agent	Yes. It must balance science value, battery and safety, and choose the best trade-off.
Learning agent	Yes. The terrain is new, so the rover improves its driving and sampling with experience.

Question 3: 8-Queens Problem – Search Space and Problem Formulation

1. Problem Understanding

We must place 8 queens on an 8 × 8 chessboard so that no queen attacks another. A queen attacks along its row, its column and both diagonals. So no two queens may share a row, a column or a diagonal.
The board in the question is not a solution because the queen in the first column and the queen in the last column are on the same diagonal. Our own example of such a board is shown below (the figure from the question was not included in the file).



2. Method Selection

Method chosen: Depth-First Search with backtracking (an uninformed search) on an incremental formulation.

- Queens are placed one column at a time, and a queen is placed only on a safe square.
- DFS goes deep quickly, and every solution lies at the same depth (8 queens), so DFS finds one fast.
- DFS keeps only the current path in memory, so it needs very little memory.
- When no safe square is left in a column, the search backtracks and tries another row.

3. Solution Process

Problem components (incremental formulation)

Component	**Definition**
States	Any arrangement of 0 to 8 queens on the board, one per column, with no queen attacking another
Initial state	Empty board – no queens placed
Actions	Add a queen to a safe square in the leftmost empty column
Transition model	Returns the board with the new queen added
Goal test	8 queens are on the board and none is attacked
Path cost	Not important – only the final arrangement matters

Size of the search space

Formulation	**Number of states**
Complete-state: 8 queens anywhere (64 C 8)	4,426,165,368
Place queens one by one on any square (64 × 63 × … × 57)	about 1.8×10^{14}

Formulation	Number of states
One queen per column, any row (8^8)	16,777,216
Incremental, safe squares only (used here)	2,057 (counted by the program)

The incremental formulation shrinks the search space enormously, which is why the search finishes instantly.

Program in VS Code and its real output

```python
def is_safe(rows, col, row):
    for c in range(col):
        r = rows[c]
        if r == row or abs(r - row) == abs(c - col):   # same row or same diagonal
            return False
    return True

def dfs(rows, col, solutions, first_only=False):
    if col == N:                    # GOAL TEST: all 8 queens placed safely
        solutions.append(rows[:])
        return first_only
    for row in range(N):            # ACTIONS: try every row in this column
        if is_safe(rows, col, row):
            rows.append(row)
            if dfs(rows, col + 1, solutions, first_only):
                return True
            rows.pop()              # backtrack
    return False

# 1) A near-solution: first and last column queens share a diagonal
bad = [0, 2, 4, 6, 1, 3, 5, 7]
print("Checking a near-solution (like the one in the question):")
print("Queen in column 1 is at row", bad[0] + 1, "| queen in column 8 is at row", bad[7] + 1)
print("Same diagonal?", abs(bad[0] - bad[7]) == 7, "-> NOT a solution")
print()

# 2) Find the first solution using depth-first search with backtracking
first = []
dfs([], 0, first, first_only=True)
print("First solution found by DFS (row number for columns 1..8):")
print(*[r + 1 for r in first[0]])
print()
for r in range(N):
    print(" ".join("Q" if first[0][c] == r else "." for c in range(N)))

# 3) Count all solutions
allsol = []
dfs([], 0, allsol)
print("\nTotal number of solutions:", len(allsol))
```

```
Checking a near-solution (like the one in the question):
Queen in column 1 is at row 1 | queen in column 8 is at row 8
Same diagonal? True -> NOT a solution

First solution found by DFS (row number for columns 1..8):
1 5 8 6 3 7 2 4

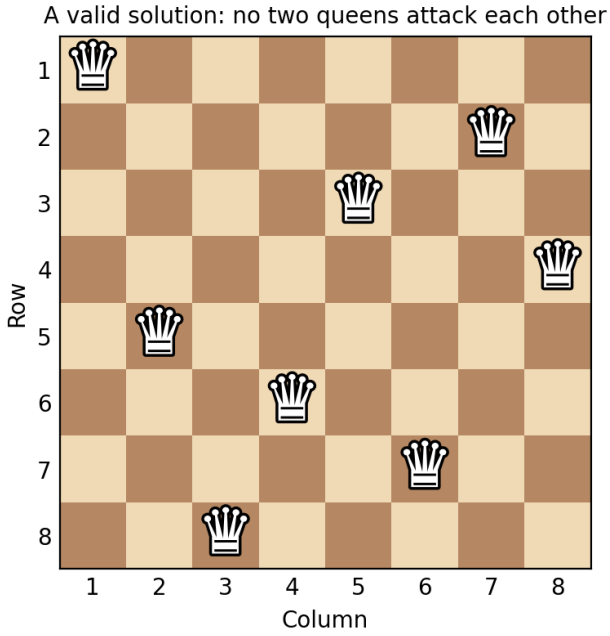
Q . . . . . .
. . . . . Q .
. . . . Q . .
. . . . . . Q
. Q . . . . .
. . . Q . . .
. . . . . Q .
. . Q . . . .

Total number of solutions: 92
```

4. Accuracy of Calculation

The values below come directly from the program output and the counting run:

- **Near-solution check:** queen in column 1 is in row 1 and queen in column 8 is in row 8, so they share a diagonal. This is not a solution.
- **First solution found by DFS (row for columns 1 to 8):** 1 5 8 6 3 7 2 4
- **Search effort:** DFS visited only 114 states before finding this solution.
- **Total number of solutions:** 92 (12 are truly different; the others are rotations or mirror images).



5. Interpretation of Results

- Good problem formulation matters more than a faster computer: the same puzzle goes from about 1.8×10^{14} states to just 2,057.
- Backtracking works because a wrong choice is cancelled as soon as it is found, so most of the board's possibilities are never explored.
- The check on the near-solution proves the goal test works: it correctly rejects a board with a diagonal clash.
- 8-Queens is an example of a constraint-satisfaction problem that uninformed search can solve.

Question 4: OLA Cab Booking – Problem-Solving Agent

1. Problem Understanding

A customer opens the OLA app, enters a pickup place and a destination, and chooses a cab type (Micro, Mini, Prime Sedan, Sedan, Shared and so on) based on comfort and price. The app must book a cab and take the customer to the destination.

Item	In this problem
Input (percepts)	Pickup location, destination, cab types available, price, traffic, customer preference
Goal	Customer reaches the destination in the chosen cab

Item	In this problem
Actions	Search nearby cabs, choose cab type, book cab, track cab, start trip, end trip, pay
Performance	Short waiting time, low fare, comfort, safe trip

2. Method Selection

Agent chosen: a Goal-Based Problem-Solving Agent (with a utility function to pick the best cab type).

- The agent has a clear goal: reach the destination.
- It follows the problem-solving cycle: formulate goal → formulate problem → search for a solution → execute the plan.
- A simple reflex agent is not enough, because the route and cab choice need planning.
- A utility score (comfort versus price) lets the agent choose between cab types that all reach the goal.

Problem formulation

Component	Definition
Initial state	Customer at the pickup location, no cab booked
Actions	Select cab type, book cab, travel along a road to the next place
Transition model	Customer moves to the next location, or the booking status changes
Goal test	Customer is at the destination
Path cost	Fare, travel time and distance

3. Solution Process

The same logic as a runnable Python program in VS Code

```
CABS = {   # type: (price per km in Rs, comfort score out of 10, seats)
    "Micro":  (9, 5, 4), "Mini": (11, 6, 4), "Prime Sedan": (14, 8, 4),
    "Sedan": (13, 7, 4), "Shared": (6, 4, 2),
}
def formulate_goal(pickup, drop):
    return {"pickup": pickup, "drop": drop}

def find_route(goal):                    # stands in for the map / search module
    return 12.0, 35                      # distance in km, time in minutes

def choose_cab(distance, preference):    # picks the cab type that suits the customer
    best, best_score = None, -1
    for name, (price, comfort, seats) in CABS.items():
        score = comfort * preference["comfort"] - price * preference["cheap"]
        if score > best_score:
            best, best_score = name, score
    return best, round(CABS[best][0] * distance)

def ola_agent(pickup, drop, preference):
    goal = formulate_goal(pickup, drop)
    km, mins = find_route(goal)
    cab, fare = choose_cab(km, preference)
    print(f"Goal      : reach {goal['drop']} from {goal['pickup']}")
    print(f"Route     : {km} km, about {mins} min")
    print(f"Cab chosen: {cab}")
    print(f"Fare      : Rs {fare}")
    print("Status    : Booking confirmed, driver on the way")

ola_agent("Guindy", "Chennai Airport", {"comfort": 3, "cheap": 1})
```

```
Goal      : reach Chennai Airport from Guindy
Route     : 12.0 km, about 35 min
Cab chosen: Prime Sedan
Fare      : Rs 168
Status    : Booking confirmed, driver on the way
```

4. Accuracy of Calculation

The program scores each cab as comfort × 3 − price per km × 1 (the customer cares more about comfort):

Cab type	Comfort (out of 10)	Price per km (Rs)	Score = 3 × comfort − price
Micro	5	9	15 − 9 = 6
Mini	6	11	18 − 11 = 7
Sedan	7	13	21 − 13 = 8
Prime Sedan	8	14	24 − 14 = 10 (best)
Shared	4	6	12 − 6 = 6

Fare check: 12 km × Rs 14 = Rs 168, which matches the program output.

5. Interpretation of Results

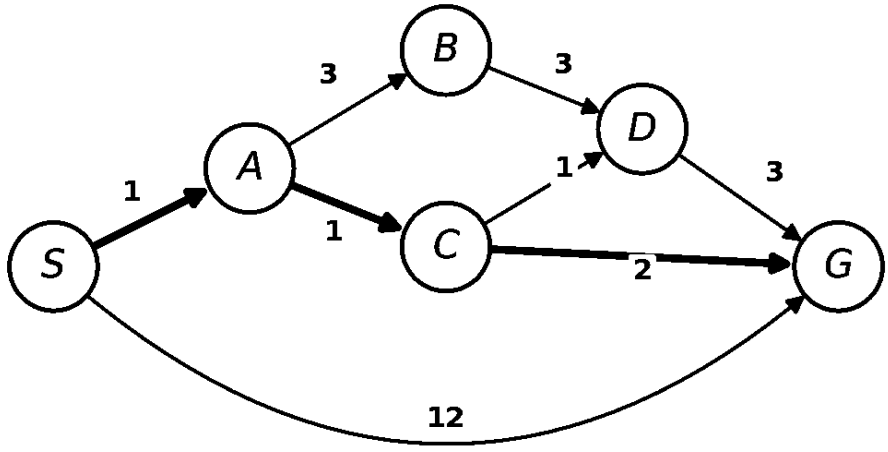
- The agent turned the customer's goal into a plan and chose Prime Sedan because it had the highest score for this customer.
- If the customer cares more about price, the weights change and the agent picks a cheaper cab, so the same agent adapts to different customers.
- A goal-based agent suits this app because there is one clear goal and several ways to reach it.

- The route distance (12 km) and time (35 min) in the program are sample values. A real app would get them from a map service.

Question 5: Uniform Cost Search – Least-Cost Delivery Path

1. Problem Understanding

A logistics company moves packages between warehouses. Each road has a cost (fuel + time). We must find the cheapest path from the start warehouse S to the destination warehouse G.



Red path S → A → C → G is the least-cost path (total cost = 4)

From	To	Cost
S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

2. Method Selection

Method chosen: Uniform Cost Search (UCS), as the question asks.

- UCS always expands the node with the lowest total cost so far, g(n), using a priority queue.
- The roads have different costs, so Breadth-First Search (which counts only the number of roads) can return a wrong, costly answer.
- UCS is complete and optimal when all costs are positive, so the first time G is removed from the queue, its path is the cheapest.
-

3. Solution Process

Algorithm in simple steps

1. Put S in the priority queue with cost 0.
2. Remove the node with the lowest cost from the queue.

3. If it is G, stop. The path and its cost are the answer.
4. Otherwise mark it explored and add each unexplored neighbour with cost = cost so far + road cost.
5. Repeat from step 2.

Program in VS Code and its real output

```
import heapq

graph = {
    "S": {"A": 1, "G": 12},
    "A": {"B": 3, "C": 1},
    "B": {"D": 3},
    "C": {"D": 1, "G": 2},
    "D": {"G": 3},
    "G": {},
}

def ucs(start, goal):
    frontier = [(0, start, [start])]       # (cost so far, node, path)
    explored = set()
    while frontier:
        cost, node, path = heapq.heappop(frontier)   # cheapest first
        print(f"Expand {node}  cost={cost}  frontier={sorted((c, n) for c, n, _ in frontier)}")
        if node == goal:
            return cost, path
        if node in explored:
            continue
        explored.add(node)
        for nxt, w in graph[node].items():
            if nxt not in explored:
                heapq.heappush(frontier, (cost + w, nxt, path + [nxt]))
    return None
```

```
Expand S  cost=0  frontier=[]
Expand A  cost=1  frontier=[(12, 'G')]
Expand C  cost=2  frontier=[(4, 'B'), (12, 'G')]
Expand D  cost=3  frontier=[(4, 'B'), (4, 'G'), (12, 'G')]
Expand B  cost=4  frontier=[(4, 'G'), (6, 'G'), (12, 'G')]
Expand G  cost=4  frontier=[(6, 'G'), (12, 'G')]
```

4. Accuracy of Calculation

Step-by-step trace (matches the program output). Notation: node (cost so far).

Step	Node expanded	New nodes added to the queue	Queue after the step (sorted)
1	S (0)	A (0+1 = 1), G (0+12 = 12)	A1, G12
2	A (1)	B (1+3 = 4), C (1+1 = 2)	C2, B4, G12
3	C (2)	D (2+1 = 3), G (2+2 = 4)	D3, B4, G4, G12
4	D (3)	G (3+3 = 6)	B4, G4, G6, G12
5	B (4)	none (D is already explored)	G4, G6, G12
6	G (4)	Goal reached – stop	–

B and G both have cost 4 in the queue. The program takes B first because the tie is broken alphabetically. This does not change the answer.

Cost of every possible path (to confirm the answer)

Path	Calculation	Total cost
S → A → C → G	1 + 1 + 2	4 (least)
S → A → C → D → G	1 + 1 + 1 + 3	6
S → A → B → D → G	1 + 3 + 3 + 3	10
S → G	12	12

5. Interpretation of Results

- **Final answer:** the least-cost path is S → A → C → G with a total cost of 4 (cost units = fuel + time).
- BFS would choose the direct road S → G because it has only one step, but that costs 12, which is three times more than UCS.
- The company saves 8 cost units (12 − 4) on this delivery.
- UCS may explore many nodes, but it guarantees the cheapest path, which is what matters for delivery cost.